TRANSPORT CONTROL PROTOCOL

CIS*3210

Fall 2026

Sajid Marhon

Based on notes for Computer Networking: A Top-Down Approach, 9th ed., by J. Kurose and K. Ross

TCP Connection



- **Connection-oriented protocol**
  - TCP requires a **handshake** before data transfer
  - Both sides initialize TCP state variables during connection establishment
- **Logical, not physical connection**
  - Unlike circuit switching, TCP connection is **logical** and exists only in the end systems
  - Intermediate devices (routers, switches) are **unaware** of TCP connections; they only see IP datagrams
- **Service features**
  - Provides full-duplex communication (data can flow both ways simultaneously)
  - Always point-to-point (one sender, one receiver)
  - Multicasting is not supported

TCP Connection



- **Client-server model**
  - The process initiating the connection = **client**, the other = **server**
  - Example in Python: clientSocket.connect((serverName, serverPort))
- **Three-way handshake**
  - Step 1: Client sends special TCP segment
  - Step 2: Server responds with another special TCP segment
  - Step 3: Client responds again (may include data)
- **Data transfer process**
  - Application sends data via socket → goes to **TCP send buffer**
  - TCP divides data into **segments** (limited by Maximum Segment Size, MSS)
  - MSS is based on the **link-layer MTU** (e.g., Ethernet MTU = 1500 bytes)
  - TCP segments are encapsulated in IP datagrams and sent to the network

TCP Connection

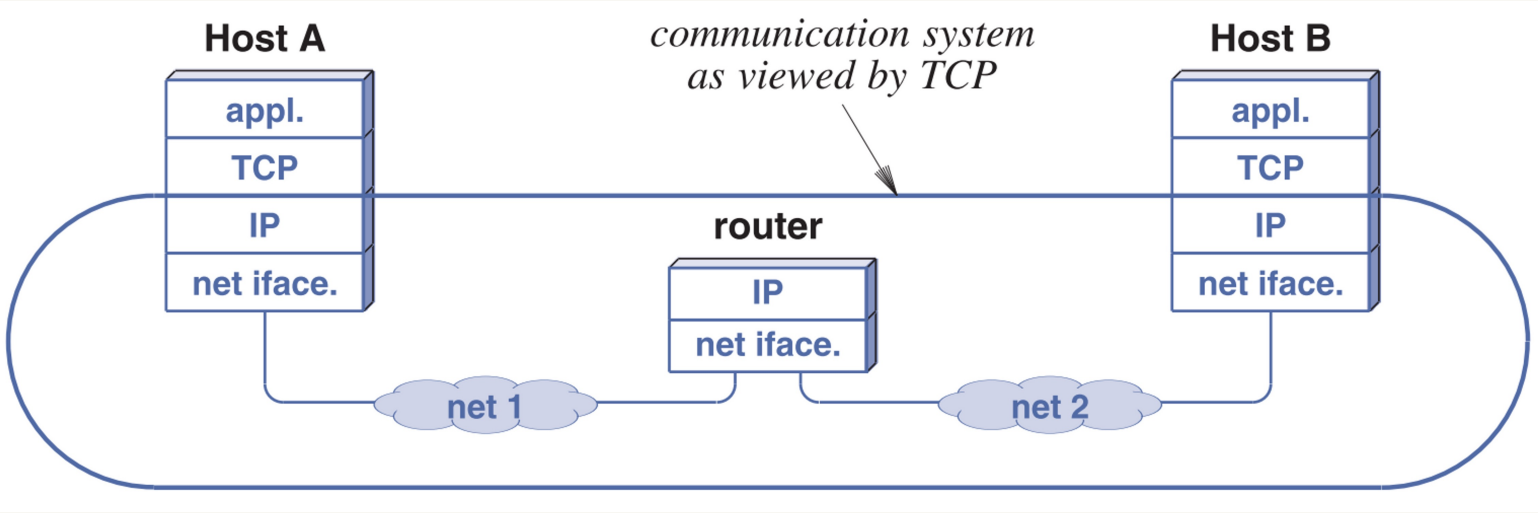


- **Receiving process**
  - At the destination, TCP places data into the **receive buffer**
  - Application reads data from the receive buffer
  - Each side maintains its **own send and receive buffers**
- **Connection components**
  - A TCP connection = **buffers, state variables, and socket endpoints** at both hosts
  - **Routers/switches/repeaters** in the middle do not store any TCP connection state

End-to-End Service and Virtual Connections

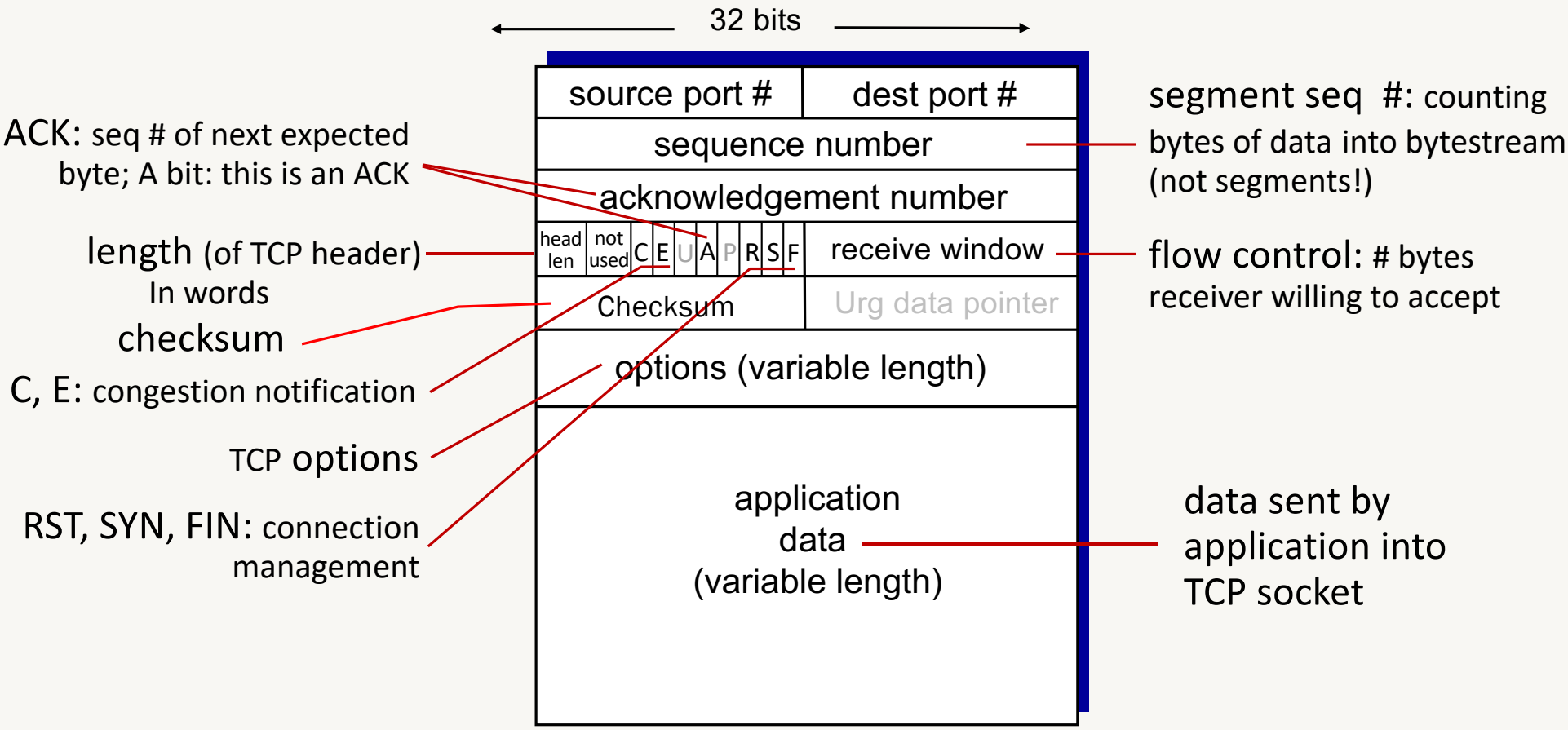


- **TCP runs only at the end hosts;** intermediate routers do not use or maintain TCP
- **From TCP's perspective**, the Internet simply delivers data end-to-end without inspecting or modifying its contents



TCP Segment Structure

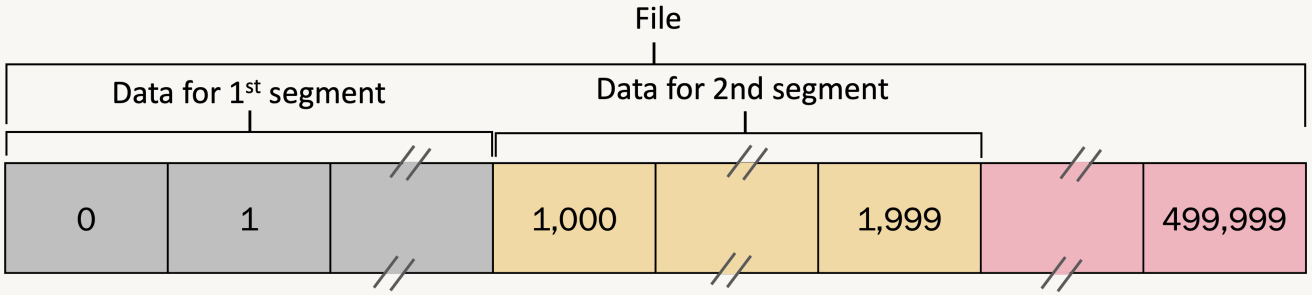




TCP Sequence numbers and ACK Numbers



- **TCP data view**
  - Treats data as an **unstructured but ordered stream of bytes**
  - Sequence numbers are assigned **per byte**, not per segment
- **Sequence numbers**
  - Sequence number in a segment = byte number of the first byte in that segment
  - Example: with MSS = 1000, first segment is 0–999 (seq# 0), second is 1000–1999 (seq# 1000), etc
  - Initial sequence number is randomly chosen (not always zero) to avoid confusion with old segments



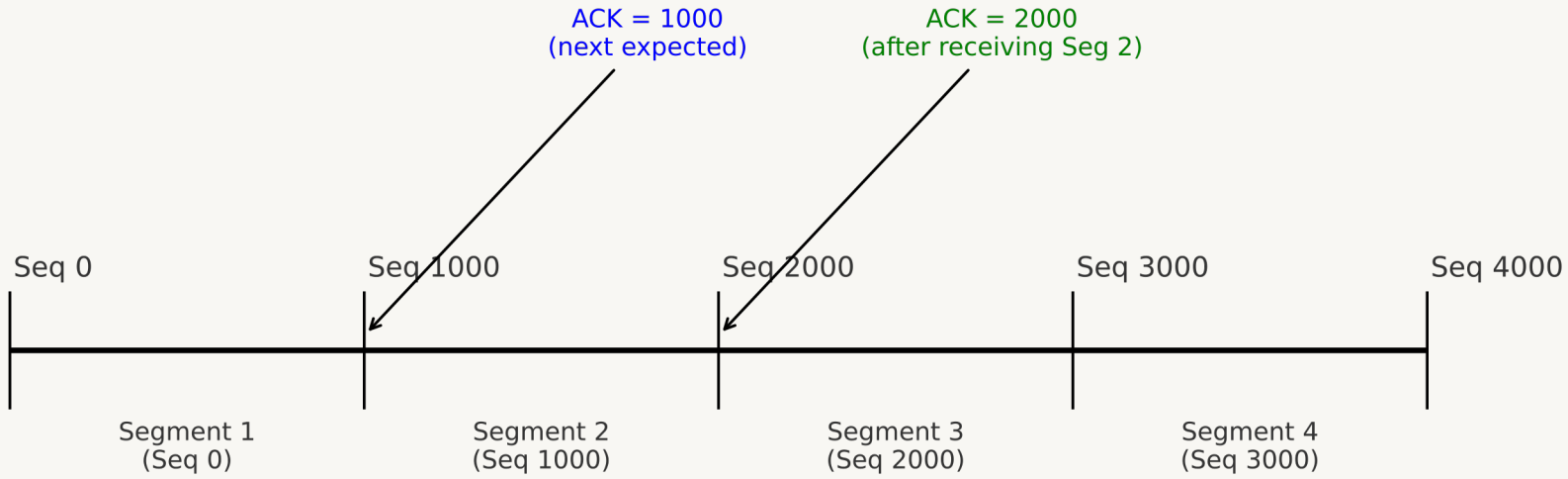
TCP Sequence numbers and ACK Numbers



- **Acknowledgment numbers**
  - ACK number = next byte expected by the receiver
  - TCP provides cumulative acknowledgments (acknowledges all bytes up to the first missing one)
  - Example: if bytes 0–535 received, ACK = 536
- **Handling out-of-order segments**
  - Two options for receiver:
    - Discard out-of-order segments
    - Store them until missing bytes arrive (more efficient, used in practice)

TCP Sequence numbers and ACK Numbers







TCP Sequence numbers and ACK Numbers

Sequence numbers:

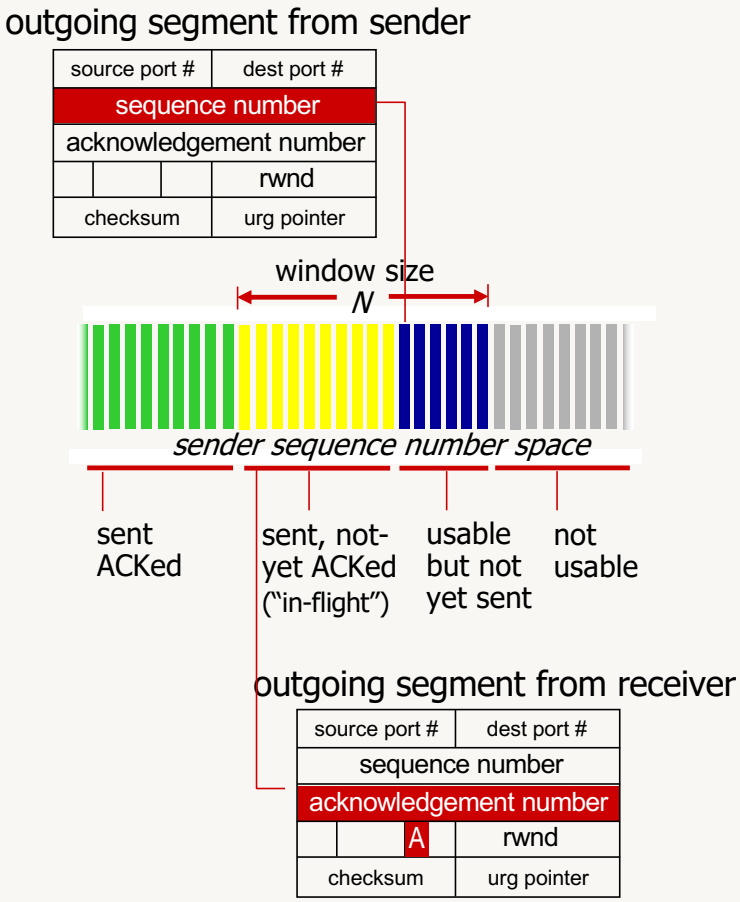
- byte stream "number" of first byte in segment's data

Acknowledgements:

- seq # of next byte expected from other side
- cumulative ACK

Q: how receiver handles out-of-order segments

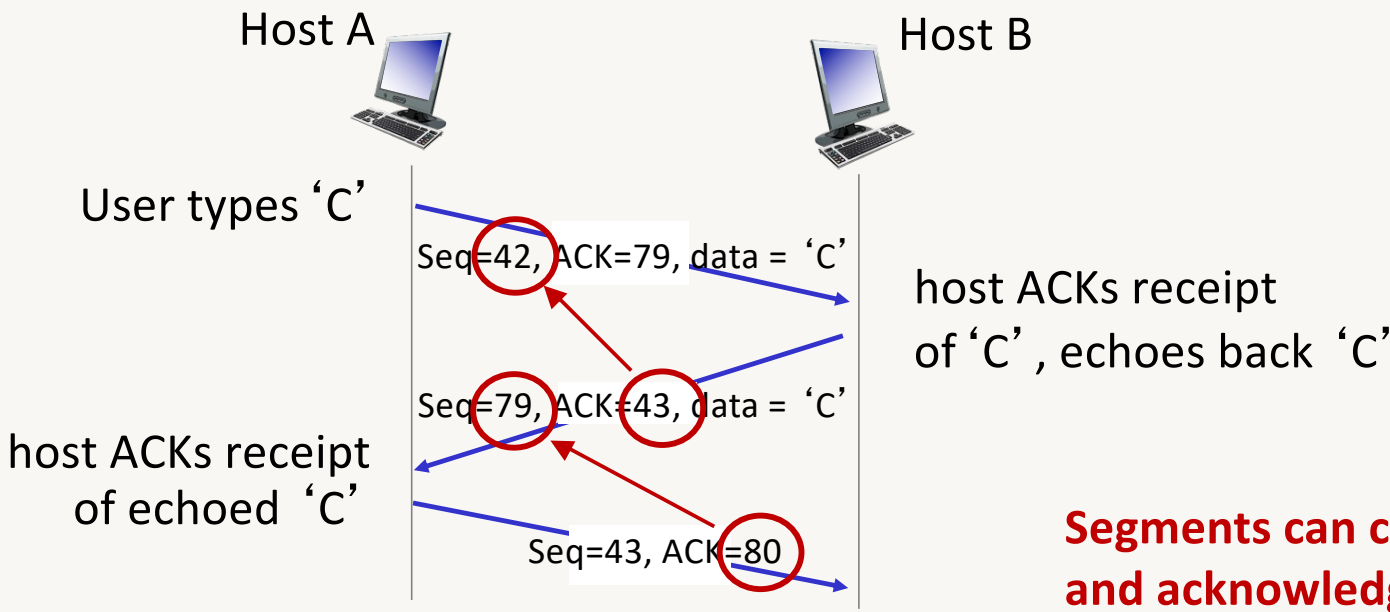
- **A:** TCP spec doesn't say, - up to implementor



TCP Sequence numbers and ACK Numbers



- **Full-duplex operation**
  - Both hosts send/receive data simultaneously, each tracking sequence numbers and ACKs for the opposite direction



Segments can carry data and acknowledge receiving data at the same time

simple telnet scenario

TCP RTT Estimation and Timeout



Q: how to set TCP timeout value?

- longer than RTT, but RTT varies!
- **too short:** premature timeout, unnecessary retransmissions
- **too long:** slow reaction to segment loss

Q: how to estimate RTT?

- **`SampleRTT`**: measured time from segment transmission until ACK receipt
  - ignore retransmissions
- **`SampleRTT`** will vary, want estimated RTT "smoother"
  - average several *recent* measurements, not just current **`SampleRTT`**

TCP RTT Estimation and Timeout



- **TCP uses timeout/retransmit** to recover from lost segments
- Timeout must be **greater than RTT** (round-trip time) to avoid unnecessary retransmissions
- **RTT measurement**
  - **SampleRTT** = time from sending a segment to receiving its ACK
  - TCP measures **only one SampleRTT at a time** (not for every segment)
  - **No RTT measurement for retransmitted segments**
- **RTT variability**
  - SampleRTT values fluctuate due to **network congestion** and **system load**
  - A single sample may not represent typical RTT

TCP RTT Estimation and Timeout

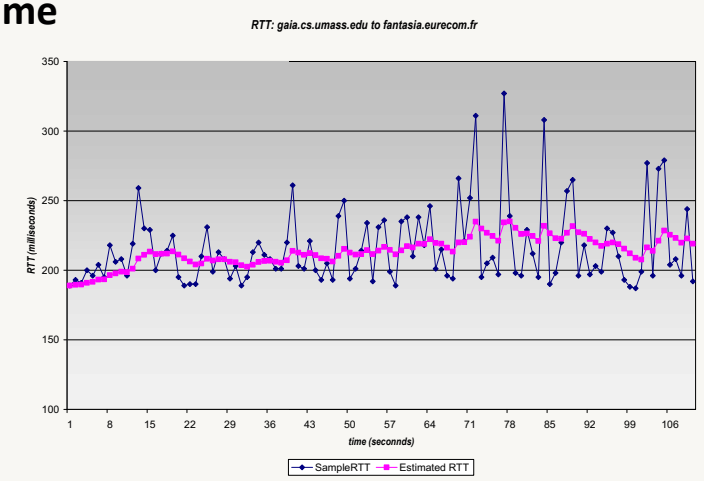


- **EstimatedRTT**
  - TCP computes a **weighted average** of SampleRTT values
  - Formula:

$$EstimatedRTT = (1 - \alpha) \times EstimatedRTT + \alpha \times SampleRTT$$

  where $\alpha = 0.125$ (1/8) according to [RFC 6298]

- **Properties of the estimate**
  - Recent RTT samples have **more influence** than older ones
  - This technique is called an Exponential Weighted Moving Average (EWMA)
  - The weight of old samples decays **exponentially over time**



TCP RTT Estimation and Timeout



- **Timeout interval basics**
  - Must be **≥ EstimatedRTT** to avoid unnecessary retransmissions
  - Should not be **too large**, or lost segments will cause long delays
- **Adding a margin**
  - Timeout interval = **EstimatedRTT + margin**
  - Margin depends on **fluctuation in RTT**:
    - Large variation ⇒ larger margin
    - Stable RTT ⇒ smaller margin
  - The DevRTT (RTT deviation) is used to adjust this margin

$$TimeoutInterval = EstimatedRTT + 4\times DevRTT$$

$$DevRTT = (1-\beta)\times DevRTT + \beta\times|SampleRTT - EstimatedRTT|, \qquad \beta = 0.25$$

- **TCP's timeout management**
  - Initial timeout = 1 second (RFC 6298)
  - If a timeout occurs, TCP doubles the timeout value
  - Once an ACK is received and EstimatedRTT is updated, TCP recomputes the timeout using the formula

TCP Sender Events



1. **Event: Data from application**
   - Encapsulated in a TCP segment with sequence number
   - Passed to IP
   - If no timer running, start timer (for oldest unacknowledged segment)
2. **Event: Timeout**
   - Retransmit the segment that caused the timeout
   - Restart the timer
3. **Event: ACK received**
   - ACK field is compared with SendBase (oldest unacknowledged byte)
   - If ACK > SendBase ⇒ new data acknowledged
   - Update SendBase
   - Restart timer if unacknowledged data remains

TCP Receiver: ACK Generation

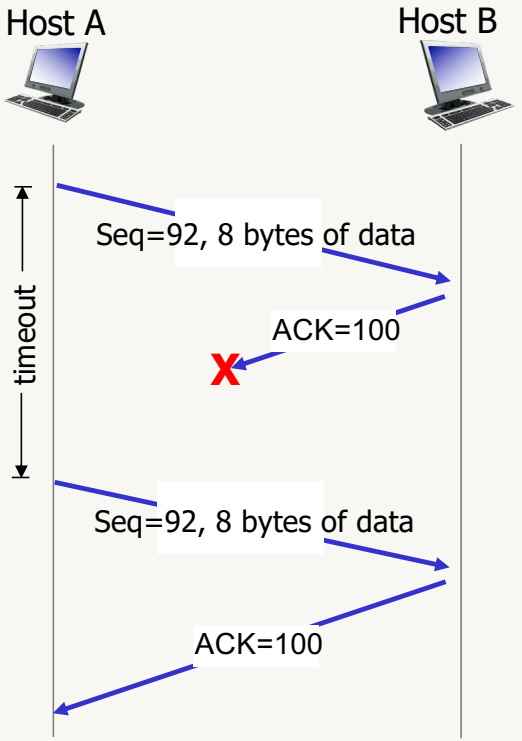


Event at receiver	TCP receiver action
arrival of in-order segment with expected seq #. All data up to expected seq # already ACKed	delayed ACK. Wait up to 500ms for next segment. If no next segment, send ACK
arrival of in-order segment with expected seq #. One other segment has ACK pending	immediately send single cumulative ACK, ACKing both in-order segments
arrival of out-of-order segment higher-than-expect seq. # . Gap detected	immediately send **duplicate ACK (acking last correctly received in order byte),** indicating seq. # of next expected byte
arrival of segment that partially or completely fills gap	immediate send ACK, provided that segment starts at lower end of gap

TCP Retransmission Scenarios



- **Scenario 1: Lost ACK**
  - Host A sends one segment (Seq=x, L bytes)
  - Host B receives it and sends ACK (x+L), but ACK is lost
  - Timeout occurs → Host A retransmits the segment
  - Host B discards retransmitted data (already received)

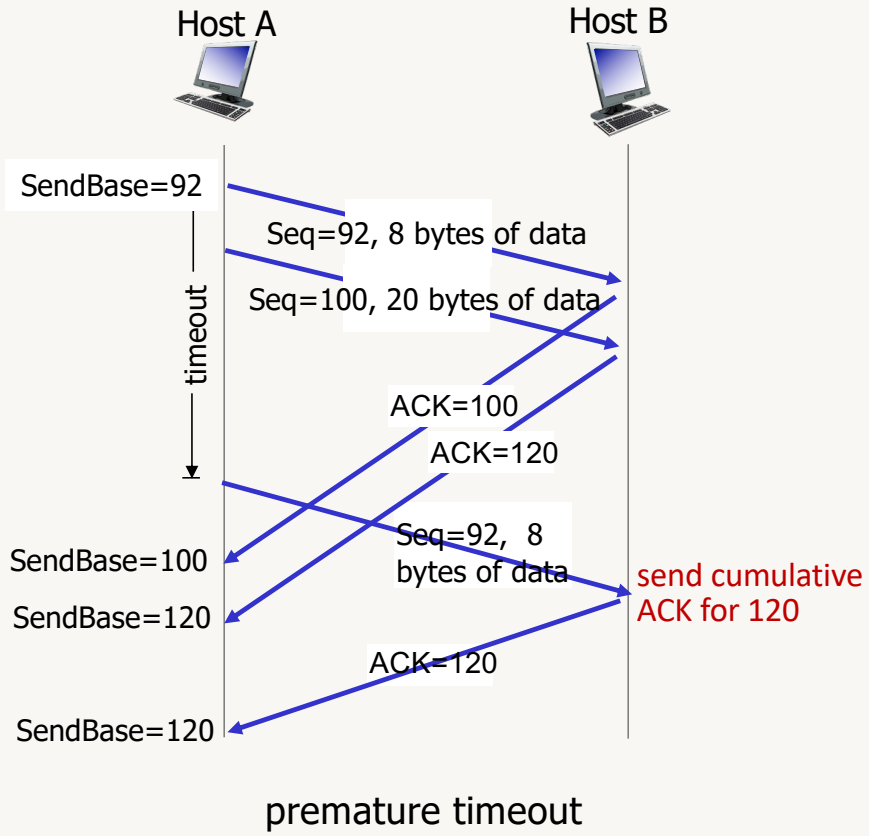


lost ACK scenario

TCP Retransmission Scenarios



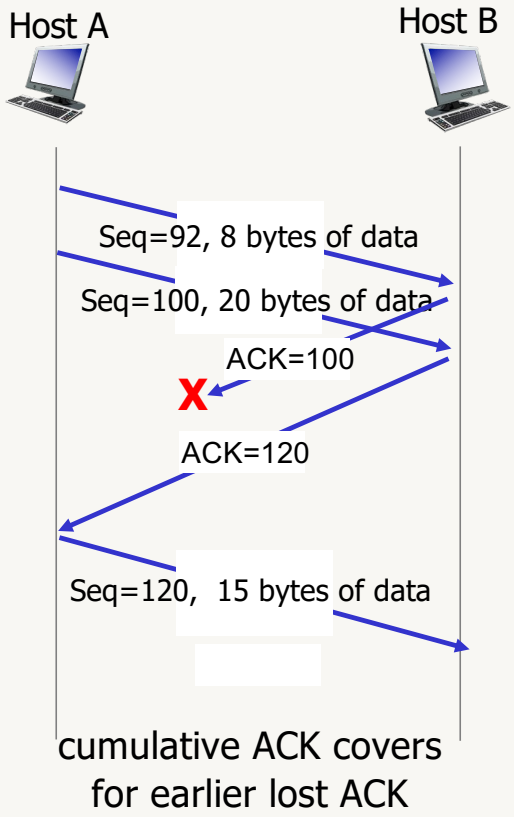
- **Scenario 2: Premature Timeout with Delayed ACKs**
- Host A sends **two back-to-back segments** (Seq = x and Seq = x+L).
- Host B receives both correctly and sends **two ACKs** (ACK = x+L and ACK = x+2L)
- The ACKs are **delayed** and **arrive after the timeout**, not lost
- Timeout occurs → Host A **retransmits only the first segment** (Seq = x)
- Timer restarts; if the **second ACK arrives before the new timeout**, the **second segment is not retransmitted**



TCP Retransmission Scenarios



- **Scenario 3: Partial ACK loss**
  - Host A sends two segments (Seq=x, Seq=x+L)
  - ACK for first segment (x+L) is lost
  - Before timeout, Host A receives ACK for second segment (x+2L)
  - This cumulative ACK confirms both segments received → Host A does not retransmit



TCP Retransmission Scenarios



TCP fast retransmit

- Sender waits for **3 duplicate ACKs** (i.e., 4 ACKs for the same data in total)
- Interprets this as a **loss of the next segment**
- **Retransmits immediately** (before timeout) ⇒ reduces delay



Receipt of three duplicate ACKs indicates 3 segments received after a missing segment – lost segment is likely. So retransmit!

